CNENGR115 Computers and Control

Computers

Lecture 3/3 – Practical assessment final instructions

Dr Zhongming Zhang

School of Engineering
Lancaster University

Embedded Systems

Lancaster University

Revision of Lecture 2:

- As Engineers we select an embedded system on the basis of the *device family* and the specific *model* (i.e. facilities) within such a family.
- An *embedded system* is a computer that steps through a sequential set of instructions in memory (a *program*), applying each to some numbers also stored in memory (*data*), storing the processed *output* in memory.
- The data can be recorded from external devices (*sensors*) or set up internally in an initialisation stage; the output is often used to control external devices (*actuators*).
- Embedded systems span the *Von Neumann* and *Harvard* computer architectures, and the *RISC* and *CISC* instruction set architectures.
- A wide variety of devices exist, c.f. the Atmel AVR family, the Motorola MC68000 family, the 8051 family, the Microchip PIC family and the Raspberry Pi.

Design brief

- Select (at least) one sensor.
- Select (at least) one actuator.
- ***"Design a system that controls the actuator(s) based on the signal(s) received by the sensor(s)."***
- **Not expected** to implement a feedback control action that compensates for disturbances (e.g. proportional, integral or proportional- integral control).

Design brief

- ***You are expected to stimulate, control or report signals to the actuator(s) based on signals and conditions interpreted from your sensor(s).***
- E.g. Lesson 16: The brightness of a LED (actuator) that is inversely proportional to the resistance of a photoresistor (sensor).

Design brief

- **Actuators**: tri-colour LED module, ultrasound module, servo motor, 7-segment display, 8x8 matrix display, LCD module, OLED display, stepper motor, LED, active buzzer, passive buzzer.
- **Sensors**: thermistor, touch module, ultrasound module, flame sensor, vibration sensor, temperature sensor, IR receiver, photoresistor, potentiometer, key switch.

Design brief criteria

- Attach your Arduino code for your design.
  - code that is structured and uses good programming practices, such as;
    - indentation,
    - whitespace,
    - comments,
    - meaningful variable names.

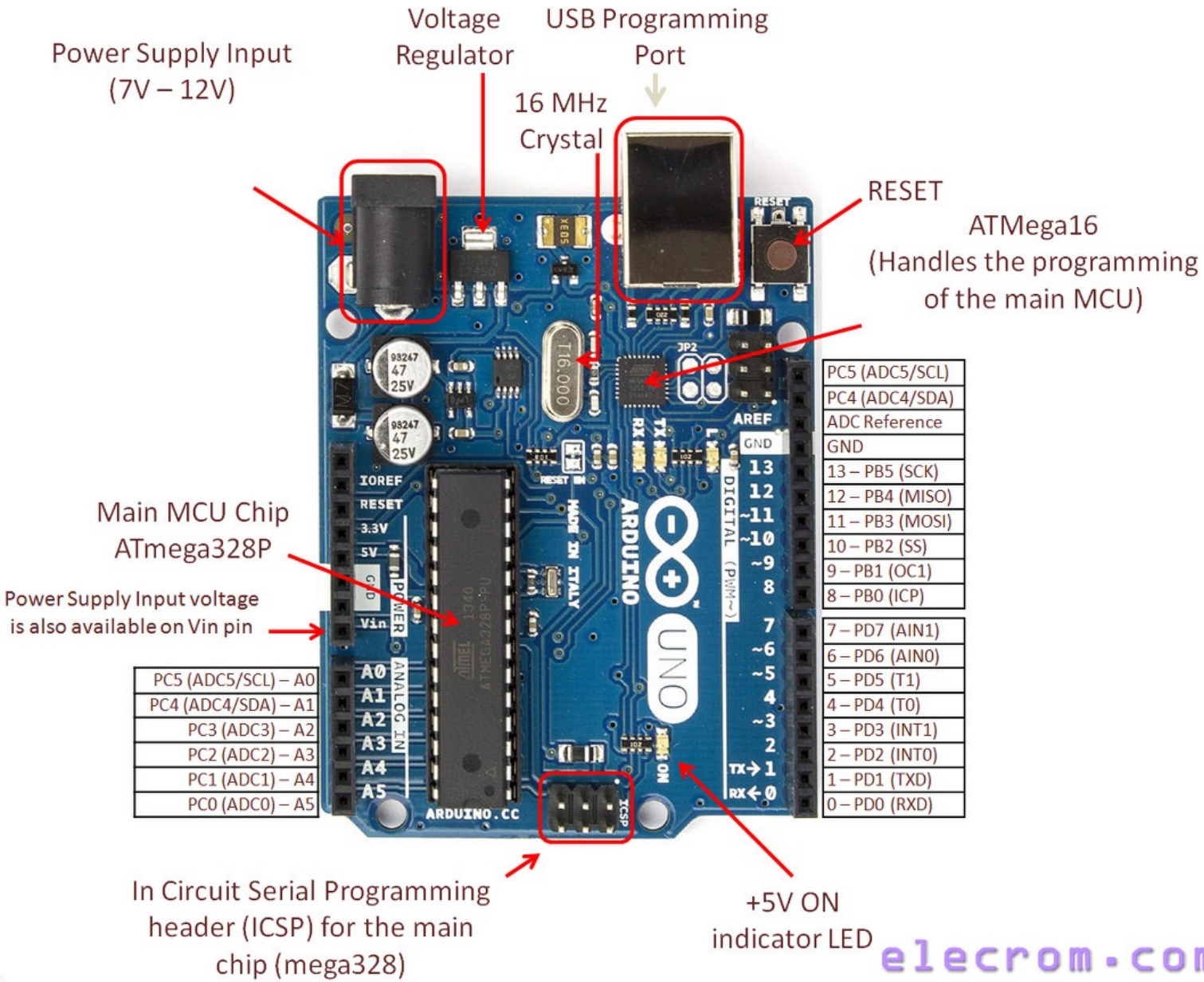
Design brief marks

- **Novelty of design** (5 marks) *Unlike to any of the practical exercises that you have done, or examples provided in the resources. You can use elements of different exercises and examples that you have seen.*
- **Complexity of design** (5 marks) *Difficulty of implemented sensors/actuators and/or number of sensors/actuators.*
- **Quality of code** (5 marks) *Logical use of the programming constructs, organised in an efficient manner, so that a future programmer could quickly understand your code and develop it further, includes concise but meaningful comments.*
- **Demonstration** (5 marks) *Brief, fault-free, working demonstration.*

Design brief weight and assessment feedback

- This assessment is worth 27% of the module.
- Feedback will be given throughout the laboratory classes (formative assessment).
- A summative assessment will be made of the final demonstration of the designed embedded system during the laboratory class

Arduino UNO



elecrom.com

Marks for the code

Blink | Arduino 1.8.5

File Edit Sketch Tools Help

Blink

```
  https://www.arduino.cc/en/Main/Products

  modified 8 May 2014
  by Scott Fitzgerald
  modified 2 Sep 2016
  by Arturo Guadalupi
  modified 8 Sep 2016
  by Colby Newman

  This example code is in the public domain.

  http://www.arduino.cc/en/Tutorial/Blink
*/

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

First comment: name, date, brief description of function

Comments

Whitespace

Initialisation

Structure

Indentation

(Meaningful variable names)

Loop

Concise meaningful comments

Example code

(Example of good practice)

photosensor_LED_PWM_v1 | Arduino 1.8.5

photosensor_LED_PWM_v1

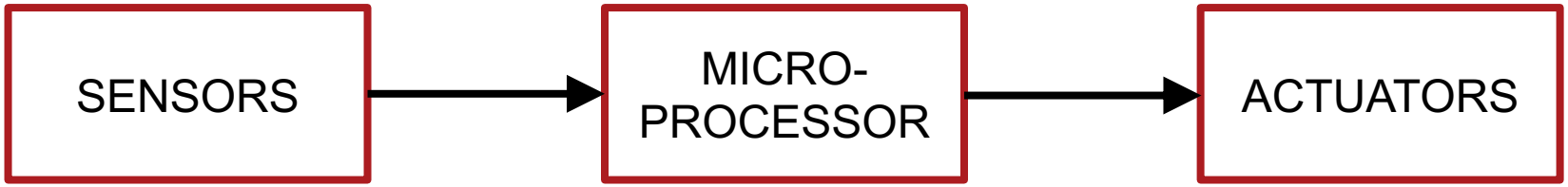
```
/* This function digitises the change in voltage produced by the change in resistance
 * of a photoresistor connected as a potential divider. This changing voltage is used
 * to control a pulse width modulated (PWM) signal that in turn controls the
 * brightness (intensity) of an LED.
 * The photoresistor's resistor is inversely proportional to the intensity of the LED.
 *
 * This system has been designed to maintain a constant light level in order to
 * optimise the growing conditions of tomato plants grown in a glasshouse.
 *
 * By M D Aspinall, 22 September 2018
*/

int phopin = 0;   // Define the analog interface 0, connected to the photosensitive resistance
int ledpin = 11;  // Define digital interface 11, PWM output to adjust LED brightness
int val = 0;      // Defined variable val

void setup()
{
  pinMode(ledpin,OUTPUT); // Define digital interface 11 as an output
  Serial.begin(9600);     // Set the baud rate to 9600
}

void loop()
{
  val = analogRead(phopin);   // Read the sensor's analogue values and assign to 'val'
  Serial.println(val);        // Display 'val' variable value
  analogWrite(ledpin, val/4); // Turn on the LED and set the brightness (the maximum value of the PWM output is 255)
  delay(10);                  // Delay 0.01 seconds
}
```

Overview of what you will implement



Ref: C520B_Arduino_Projects_by_Chumbaka.pdf

sensor_sig ∝ sig_actuator

sensor_sig ∝ 1/sig_actuator

sensor_on = on_actuator

sensor_on = off_actuator

sensor_range = on-off_actuator

sensor sig {converted to} sig actuator

Actuators recap

(All digital so no need to worry about DAC)

- Tri-colour LED module
- Ultrasound module
- Servo motor
- 7-segment display
- 8x8 matrix display
- LCD module
- OLED display
- Stepper motor
- LED
- Active buzzer
- Passive buzzer

- These all **require** a digital signal
  - Some are simply ON or OFF, single or multiple digital lines.
  - Some can be varied using a pulse train or Pulse Width Modulation (PWM) or frequency modulation or encoded data (communication protocols, e.g. SPI or I2C).

Sensors recap

(Some digital, some analogue so need ADC)

- Thermistor
- (Flame sensor)
- Temperature sensor
- Photoresistor
- Potentiometer

- These all **provide** an analogue signal
  - A varying resistance that induces a varying voltage.

- Touch module
- Ultrasound module
- Vibration (tilt) sensor
- IR receiver
- Key switch

- These all **provide** a digital signal
  - Some are simply ON or OFF.
  - Some produce a pulse width modulated signals or encoded data.
  - Simply use the digital I/Os, the Arduino Uno has 14 (of which 6 provide PWM output).

Recap – if you use a sensor that **provides** an analogue signal?

- The Arduino Uno **has six** built in ADCs!
  - Resolution 10 bit (2^{10} = 1024).
  - Therefore, the **raw values** from the ADC that you observe using the Serial Plotter will range from 0 – 1023.
  - This only represents the varied voltage on the ADC input, you have to convert it into **something meaningful** (e.g. temperature, speed, ventilation rate….).
  - Check out https://store.arduino.cc/arduino-uno-rev3

Summary

- Select (at least) one sensor.
- Select (at least) one actuator.
- ***"Design a system that controls the actuator(s) based on the signal(s) received by the sensor(s)."***
  - sensor_sig ∝ sig_actuator
  - sensor_sig ∝ 1/sig_actuator
  - sensor_on = on_actuator
  - sensor_on = off_actuator
  - sensor_range = on-off_actuator
  - sensor sig {converted to} sig actuator